

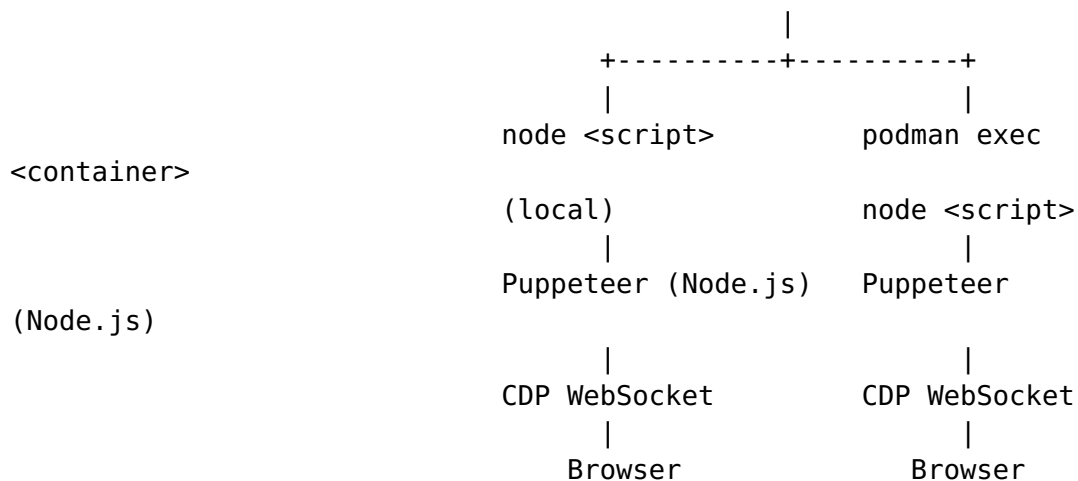
puppeteer-adapter

Puppeteer Adapter

The Puppeteer adapter implements BrowserAdapter by executing Puppeteer via Node.js scripts. Each browser action generates a Node.js script, writes it to a temporary file, and runs it via node. If local execution fails, the adapter falls back to container execution via podman exec.

Architecture

Go Challenge --> PuppeteerAdapter --> Write .js script to temp file



Two-Phase Execution

1. **Launch phase** (Initialize): Calls `puppeteer.launch()` to start a browser instance, captures the WebSocket endpoint, then disconnects
2. **Operation phase** (all other methods): Calls `puppeteer.connect({ browserWSEndpoint })` to reattach to the running browser, performs the action, then disconnects

This means the browser runs as a long-lived process, while each Go method creates a short-lived Node.js process that connects to it. The WebSocket endpoint is the coordination point.

Container Fallback

When local node execution fails (e.g., Node.js not installed, Puppeteer not available), the adapter falls back to podman exec:

1. Copies the script file into the container via `podman cp`
2. Executes it via `podman exec <container> node <script>`
3. If `podman cp` fails, falls back to inline execution: `podman exec <container> node -e "<script>"`

PuppeteerOption Reference

The adapter uses functional options for configuration:

type PuppeteerOption **func**(*PuppeteerAdapter)

Option	Description	Default
WithHeadless(bool)	Run browser in headless mode	true
WithBrowserPath(string)	Path to browser executable	(Puppeteer default)
WithContainerName(string)	Container name for fallback execution	"puppeteer"

Constructor

```
// Minimal: headless with container fallback
adapter := userflow.NewPuppeteerAdapter()
```

```
// Fully configured
adapter := userflow.NewPuppeteerAdapter(
    userflow.WithHeadless(true),
    userflow.WithBrowserPath("/usr/bin/chromium"),
    userflow.WithContainerName("my-puppeteer-container"),
)
```

Default state: headless=true, containerName="puppeteer", width=1920, height=1080.

Configuration

Initialize

```
err := adapter.Initialize(ctx, userflow.BrowserConfig{
    BrowserType: "chrome",
    Headless:    true,
    WindowSize: [2]int{1920, 1080},
})
```

Launches a Chromium browser via `puppeteer.launch()` with `--no-sandbox` and `--disable-setuid-sandbox` flags (required for container environments). Captures and stores the WebSocket endpoint.

API Reference

Navigate

```
err := adapter.Navigate(ctx, "https://example.com")
```

Connects to the browser, gets the first page (or creates one), sets viewport dimensions, and navigates to the URL.

Click, Fill, SelectOption

```
err := adapter.Click(ctx, "#submit-button")
err = adapter.Fill(ctx, "#email", "user@example.com")
err = adapter.SelectOption(ctx, "#country", "US")
```

- `Fill` triple-clicks the element first to select all existing text, then types the new value
- `SelectOption` uses `page.select(selector, value)` (native Puppeteer select support)

IsVisible

```
visible, err := adapter.IsVisible(ctx, ".alert-success")
```

Finds the element via `page.$()` and checks if `boundingBox()` returns non-null. An element with no bounding box is considered invisible.

WaitForSelector

```
err := adapter.WaitForSelector(ctx, ".dashboard", 10*time.Second)
```

Uses `page.waitForSelector(selector, { timeout: ms })`.

GetText, GetAttribute

```
text, err := adapter.GetText(ctx, "h1.title")
href, err := adapter.GetAttribute(ctx, "a.link", "href")
```

Both use `page.evaluate()` to extract the value from the DOM. Results are output as JSON to stdout and parsed by the adapter.

Screenshot

```
png, err := adapter.Screenshot(ctx)
```

Uses `page.screenshot({ encoding: 'base64' })`. The base64 string is output to stdout and decoded by the adapter.

EvaluateJS

```
result, err := adapter.EvaluateJS(ctx, "return document.title")
```

The script is base64-encoded, decoded in the generated Node.js script via `Buffer.from(encoded, 'base64')`, and passed to `page.evaluate()`. Results are JSON-stringified and returned.

NetworkIntercept

```
err := adapter.NetworkIntercept(ctx, "**/api/*", handler)
// Returns nil; handler will NOT fire.
```

Not supported because each method runs in a separate Node.js process. Puppeteer's `page.setRequestInterception()` requires a persistent process. Returns nil without error.

Close, Available

```
err := adapter.Close(ctx)           // Connects and calls
    browser.close()
ok := adapter.Available(ctx)        // Runs: node -e
    "require('puppeteer')"
```

CDP Endpoint vs Launch Mode

The adapter always uses **launch mode** during Initialize:

- **Launch mode:** `puppeteer.launch()` starts a new browser process. The adapter stores the WebSocket endpoint. Best for testing where you want a fresh browser.
- **Connect mode:** If you have an existing browser running with `--remote-debugging-port=9222`, you could extend the adapter to

use `puppeteer.connect({ browserWSEndpoint: 'ws://...' })` directly.

The current implementation launches a new browser and then uses connect mode for all subsequent operations, combining the benefits of both approaches.

Container Execution Mode

When running in environments without a local Node.js installation:

```
adapter := userflow.NewPuppeteerAdapter(  
    userflow.WithContainerName("my-puppeteer"),  
)
```

The container must have:

- Node.js installed
- puppeteer npm package installed
- A browser binary (Chromium is bundled with Puppeteer by default)

Example Dockerfile:

```
FROM node:20-slim  
RUN npx puppeteer browsers install chrome  
WORKDIR /app  
RUN npm init -y && npm install puppeteer
```

Limitations

Aspect	Detail
Performance	Each method spawns a node process. Faster than Cypress (no full Cypress runner) but slower than persistent connection approaches
Browser support	Chromium only. Firefox support is experimental in Puppeteer
Network interception	Not available due to process-per-call architecture
State	Browser state persists across calls (same browser instance). Page state is maintained
Error handling	Container fallback adds latency. Inline -e fallback has shell escaping limitations

Integration with Challenge Templates

```
adapter := userflow.NewPuppeteerAdapter(  
    userflow.WithHeadless(true),  
    userflow.WithContainerName("puppeteer"),  
)  
  
challenge := userflow.NewBrowserFlowChallenge(  
    "CH-PUPPET-001",  
    "Puppeteer Checkout Flow",  
    "Verify checkout flow via Puppeteer",  
    nil,  
    adapter,  
    userflow.BrowserFlow{  
        Name: "puppeteer-checkout",  
        StartURL: "http://localhost:3000/shop",  
        Config: userflow.BrowserConfig{  
            BrowserType: "chrome",  
            Headless: true,  
            WindowSize: [2]int{1920, 1080},  
        },  
        Steps: []userflow.BrowserStep{  
            {Name: "add-item", Action: "click", Selector: ".add-to-cart"},  
            {Name: "go-cart", Action: "click", Selector: ".cart-icon"},  
            {Name: "checkout", Action: "click", Selector: "#checkout-btn"},  
            {Name: "verify", Action: "assert_visible", Selector: ".order-confirmation"},  
        },  
    },  
)
```

Source Files

- Interface: pkg/userflow/adapter_browser.go
- Implementation: pkg/userflow/puppeteer_adapter.go